

AI Overlay Assistant for Coding Help and Meeting Notes

Project Proposal for UCS503P

Submitted to: Ms. Paramveer Kaur

Thapar Institute of Engineering and Technology

Sangam Sangwan, CSED Samyak Jain, CSED

1 Higher-Order Goal

This project aims to save students time in two ordinary situations: getting stuck on a coding problem, and writing up notes after a meeting. The broader motivation is to reduce small, recurring time losses in day-to-day academic and project work. The immediate engineering objective is a desktop overlay assistant that (a) gives a short hint when a student is stuck on a coding problem on their own screen, and (b) turns a meeting's audio into a short text summary.

2 Time-to-Value

- **We will build one feature at a time:** the coding-hint feature will be built and tested first, since it needs a simpler capture path (a single screenshot) than continuous meeting audio.
- **We will start with the simplest working capture:** a hotkey that grabs the current screen, before attempting continuous or background capture.
- **We will keep the first version small:** a single working flow (trigger a hint, get a hint back) is the target for the first iteration, not a finished product.

3 Problem Statement

Two small but common problems motivate this project:

- Coding: when a student gets an error or gets stuck, they usually search the web or a forum for help. This works, but takes time and often returns answers that don't match their exact code.
- Meetings: after a project meeting, someone has to write up what was discussed and what needs to be done. This is often skipped or done incompletely, since it takes extra time after the meeting is already over.

Neither problem is severe, but both are frequent enough that a small tool addressing them is a reasonable, testable engineering project.

4 Proposed Solution

4.1 Overview

A desktop overlay assistant with two independent modes:

- Coding help: while working on a problem, the student triggers the overlay with a hotkey; it captures the current screen and returns a short hint (not a full solution).
- Meeting notes: the student turns on audio capture for a meeting; the assistant transcribes it and produces a short summary with any action items once the meeting ends.

Both modes share our own backend for hint and summary generation and logging. The overlay client itself — the floating panel, screen capture, and audio capture — is built by us for this project.

4.2 Core Workflow

Coding help:

- Student presses a hotkey while stuck; the overlay captures the current screen.
- Our backend sends the captured content to an LLM API with a prompt that asks for a short hint, not the full answer.
- Student marks whether the hint was useful.

Meeting notes:

- Student turns on audio capture (microphone and system audio) for a meeting.
- Audio is transcribed while capture is on; our backend builds a running summary.
- At the end of the session, the student receives a short summary with any action items, which they can view and download.

4.3 Operational Constraints

- For this project, both continuous capture and any hidden-display behaviour are scoped to personal, individual use — a student's own coding practice and their own meetings — not proctored exams, interviews, or another party's call without their knowledge.
- The overlay keeps a visible on/off indicator and an explicit consent step before any capture starts.
- Usable on a normal laptop; requires OS-level screen-recording and microphone permissions.
- Built using existing, off-the-shelf APIs for language and speech processing with a bring-your-own API key; the overlay window and capture logic are our own implementation.

5 Solution Approach

This is an engineering course project, so the focus is on building a working, well-tested tool using existing components for language and speech processing, not on research or new algorithms.

5.1 Hint Generation Approach

- Hints are generated by calling an existing large language model API with a prompt that asks for a short hint, not a complete solution.
- This is a deliberate, simple design choice — it keeps the tool useful for learning rather than for skipping the problem — and is implemented with prompt wording and length limits, not new algorithms.

5.2 System Architecture

A client-server architecture, built for this project:

- Client: our own Electron-based overlay (floating panel, screen capture, audio capture, and an optional hidden-display mode).
- Backend API: our own service that receives captured screen content or audio, calls the LLM and speech-to-text APIs, and returns hints or summaries.
- Database: stores users, hint/summary logs, and consent records for evaluation.

5.3 CI/CD Approach

CI/CD will be used to:

- Automatically build and run tests on every change, for both the client and the backend.

- Produce repeatable deployment artifacts to staging.
- Enable safe and frequent releases of incremental features.

6 Evaluation Criteria

6.1 Primary Evaluation Metric

Time-to-hint (TTH): time between a student triggering a hint request and receiving a hint in the overlay.

- Target: median TTH $\leq$ 1 minute during pilot.
- Attribution: measured from client/server timestamps (trigger vs. hint returned).

6.2 Secondary Evaluation Metrics

- Hint usefulness: percentage of hints marked useful by students (simple thumbs up/down).
- Summary usefulness: student rating (1–5) of whether the meeting summary captured the discussion accurately.
- Reliability: uptime of the staging/production backend during business hours during the pilot.

6.3 Pilot Validation Plan

- Ask 8–10 classmates to use the coding-help mode on their own real assignment problems over one to two weeks.
- Ask 2–3 project groups to try the meeting-notes mode on their own meeting.
- Collect the ratings above and a few short comments to guide the next iteration.

7 Scalability

1. Scalable (theoretically): the backend is stateless and decoupled from the client, so more users can be served by running more backend instances; database queries for past hints/summaries are indexed and paginated.

2. Operationally deployable (practically): the backend runs on standard web hosting with a managed database, a containerized service, and a CI/CD pipeline for staging and production.

8 Technology Stack

A mature set of tooling is available to implement this project:

- A desktop application framework (e.g. Electron) for building the overlay client.
- Web framework for our own backend REST API.
- Managed relational database.
- An existing LLM API for hint and summary generation.
- An existing speech-to-text API for transcribing meeting audio.
- Authentication approach (token-based or session-based).
- Test framework for unit/integration tests.
- CI runner and staging deployment target.

Using existing APIs for language and speech processing lets us focus our own engineering effort on the overlay client, backend, prompt design, and evaluation, rather than on model training or infrastructure that already exists as a mature service.

9 Project Scope and Deliverables

9.1 Initial Deliverable

- A minimal overlay client: floating panel plus a hotkey-triggered screenshot capture.
- Coding-help mode: hotkey-triggered hint request, short hint returned, marked useful or not.
- Basic user accounts, a consent step before capture starts, and session logging.
- Basic logging and timestamps for the TTH metric.
- CI: build + backend unit tests + linting.
- CD to staging with a single command or pipeline job.

9.2 Subsequent Deliverables

- Continuous audio capture and the meeting-notes mode (transcript through to summary).
- Optional hidden-display mode for the overlay, if time allows.
- Simple dashboard showing hint usefulness and summary ratings over time.
- Minor refinements based on pilot feedback (e.g. hint length, prompt wording).

10 Risks and Mitigations

- **Misuse in exams or interviews:** continuous capture and any hidden-display mode could, in principle, be used to hide AI assistance during a proctored exam or interview. We scope this project's use to personal practice and the student's own meetings only, require an explicit consent step before capture starts, and log every session for the user's own review.
- **Capture and overlay behaviour is nontrivial to build:** screen capture, audio capture, and hidden-window behaviour can behave differently across OS versions and are more work than they first appear. We scope the first iteration to a single hotkey-triggered screenshot, and treat continuous capture and hidden-display mode as later, optional deliverables rather than promising them upfront.
- **Over-reliance on hints:** keep hints short and incomplete by design, so the tool supports rather than replaces problem-solving.
- **Data privacy:** only process content captured during an active, consented session; do not store screenshots or audio longer than needed to generate a hint or summary; keep summaries private to the user.
- **API cost or downtime:** add basic rate limits per user and a fallback message if the LLM or speech API is unavailable.
- **Low pilot participation:** recruit pilot users from within our own course/project group first, where participation is easier to arrange.
- **Evaluation ambiguity:** define “useful hint” and “accurate summary” as simple rating questions before development begins, so results are easy to compare.

11 Summary

This project proposes a small, two-mode desktop overlay assistant — a coding-hint helper and a meeting-notes generator — built as an original implementation using existing language and speech APIs rather than new research. It follows the course criteria by shipping a narrow first iteration quickly, using CI/CD for safe releases, and evaluating itself with simple, measurable metrics (time-to-hint and usefulness ratings). Capture

and display behaviour that is harder to build (continuous audio capture, hidden-display mode) is deliberately phased into later deliverables, and their use is scoped to personal practice with consent and logging built in.